



softITo

İSTANBUL TİCARET ODASI
SoftITo Yazılım Bilişim Akademisi
Backend Developer Bootcamp

E-Ticaret Bitirme Projesi

Hazırlayan: Metehan Dünder

İletişim: metehanndundar@hotmail.com

E-Ticaret Projesi Dökümantasyonu

Bu dokümanda öncelikle projenin yapısı ve teknik detayları ele alınmıştır. Ardından, projeyi adım adım nasıl çalıştırabileceğiniz ayrıntılı bir şekilde açıklanmıştır.

1. Projenin Genel Yapısı

Bu proje, bir E-Ticaret uygulaması olup **ASP.NET Core MVC** ile geliştirilmiştir. Entity Framework Core kullanılarak veritabanı işlemleri gerçekleştirilmiştir. **Katmanlı mimari** benimsenmiş ve bu sayede de projenin daha düzenli, test edilebilir ve sürdürülebilir olması sağlanmıştır.

2. Katmanlar ve İçeriği

Proje 3 ana katmandan oluşmaktadır:

- **ETicaretBusinessKatmanı (İş Mantığı)**
- **ETicaretDalKatmanı (Veri Erişim)**
- **ETicaretDataKatmanı (Veri Modeli)**
- **ETicaretUIKatmanı (Kullanıcı Arayüzü - MVC)**

3. Katmanların Ayrıntılı Açıklamaları

3.1 ETicaretBusinessKatmanı (İş Mantığı)

Bu katman, iş mantığını içerir. Veri erişim katmanı (DAL) UI katmanı arasında bir köprü görevi görür. Burada, projeye özel iş kuralları ve validasyonlar yer almaktadır.

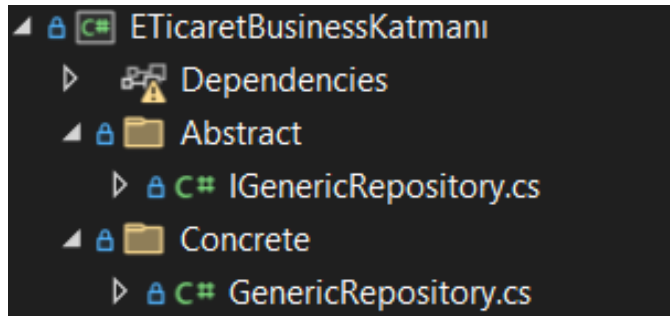
Klasör ve Dosya Açıklamaları

Abstract:

- **IGenericRepository.cs:** Tüm veritabanı işlemlerinde ortak olarak kullanılacak **generic repository arayüzüdür**. Soyut bir sınıftır.

Concrete:

- **GenericRepository.cs:** IGenericRepository arayüzünü **uygulayan somut sınıftır**.



3.2 ETicaretDalKatmanı (Veri Erişim Katmanı)

Bu katman, veritabanı işlemlerini yöneten katmandır. Entity Framework Core (EF Core) kullanılarak CRUD işlemleri (Create, Read, Update, Delete) **GenericRepository**'den miras alınarak gerçekleştirilmiştir.

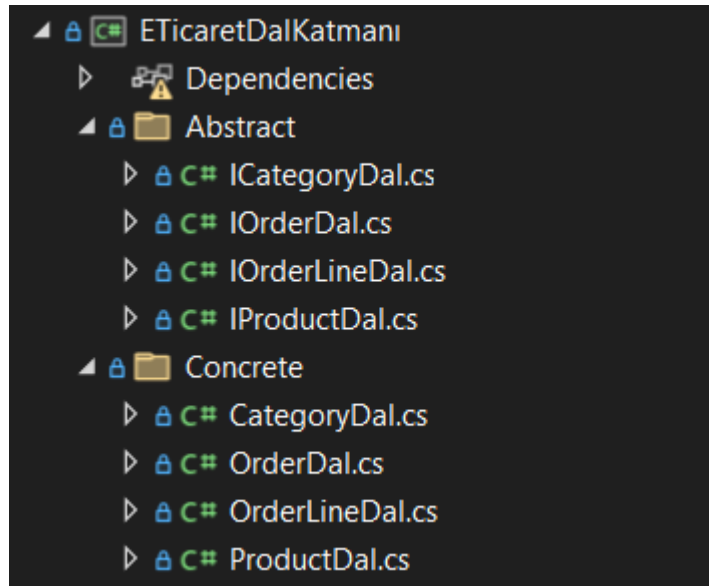
Klasör ve Dosya Açıklamaları

Abstract

- **ICategoryDal.cs:** Kategori veri erişim işlemlerini yöneten arayüz.
- **IOrderDal.cs:** Sipariş veri erişim işlemlerini yöneten arayüz.
- **IOrderLineDal.cs:** Sipariş içeriği veri erişim işlemlerini yöneten arayüz.
- **IProductDal.cs:** Ürün veri erişim işlemlerini yöneten arayüz.

Concrete

- **CategoryDal.cs:** Kategori işlemlerini yöneten sınıf (EF Core ile bağlantı).
- **OrderDal.cs:** Sipariş işlemlerini yöneten sınıf.
- **OrderLineDal.cs:** Sipariş içeriği işlemlerini yöneten sınıf.
- **ProductDal.cs:** Ürün işlemlerini yöneten sınıf.



3.3 ETicaretDataKatmanı (Veri Modeli)

Bu katman, veritabanı tablolarını temsil eden model sınıfları ve yardımcı sınıfları içerir.

Klasör ve Dosya Açıklamaları

Context

- **Context.cs:** Veritabanı bağlantısı ve DbSet tanımlamalarının yapıldığı ana sınıftır.

Entities (Varlık Modelleri): Entity'ler aslında veri tabanındaki tablolara karşılık gelen sınıflardır.

- **Category.cs:** Kategorileri temsil eden sınıf.
- **Order.cs:** Siparişleri temsil eden sınıf.
- **OrderLine.cs:** Sipariş detaylarını (sipariş içeriğini) temsil eden sınıf.
- **Product.cs:** Ürünleri temsil eden sınıf.

Helpers

- **SessionHelper.cs:** Kullanıcı **session yönetimi** için yazılmış yardımcı sınıf.

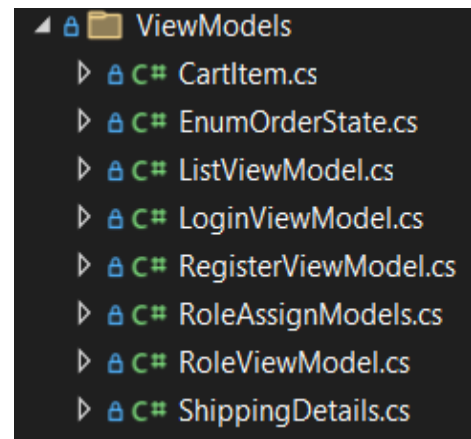
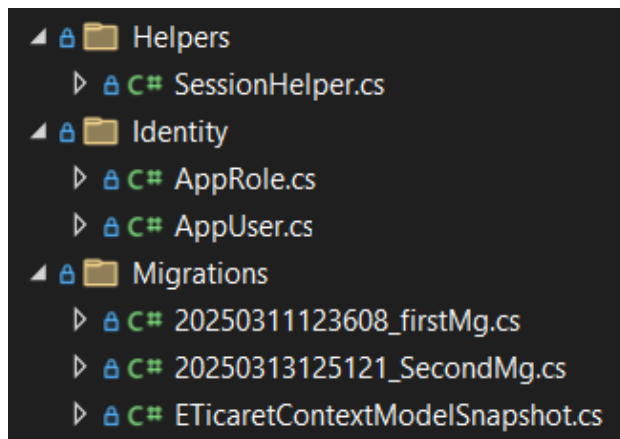
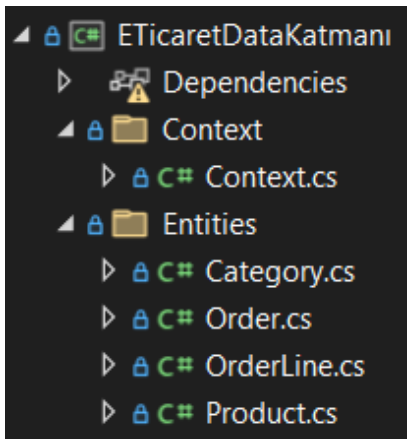
Identity (Kullanıcı Yönetimi)

- **AppRole.cs:** Kullanıcı rollerini tanımlayan model. (IdentityRole'dan türetilmiştir.)
- **AppUser.cs:** Kullanıcıları tanımlayan model (IdentityUser'dan türetilmiştir).

Migrations (Veritabanı Güncellemeleri): Veri tabanındaki değişiklikleri tutan klasör.

ViewModels (Veri Transfer Objeleri): ViewModel'lar aslında birden fazla sınıftan veri almak veya belirli alanları sınırlamak amacıyla kullanılan veri taşıma sınıflarıdır. Kullanıcıdan veri alırken veya bir tabloya ait tüm verileri getirmek istemediğimiz durumlarda ViewModel oluşturulur ve yalnızca gerekli alanlarla işlem yapılır. Böylece veri güvenliği ve performans açısından daha verimli bir yapı sağlanır.

- **CartItem.cs:** Sepet içeriğini temsil eden sınıf.
- **EnumOrderState.cs:** Sipariş durumlarını temsil eden enum.
- **LoginViewModel.cs:** Kullanıcı giriş işlemleri için model.
- **RegisterViewModel.cs:** Kullanıcı kayıt işlemleri için model.
- **ShippingDetails.cs:** Kullanıcı sipariş adresi bilgilerini içeren model.



3.4 ETicaretUIKatmanı (Kullanıcı Arayüzü - MVC)

Bu katman, kullanıcıya gösterilecek arayüzü ve MVC işlemlerini yönetir.

Klasör ve Dosya Açıklamaları

wwwroot

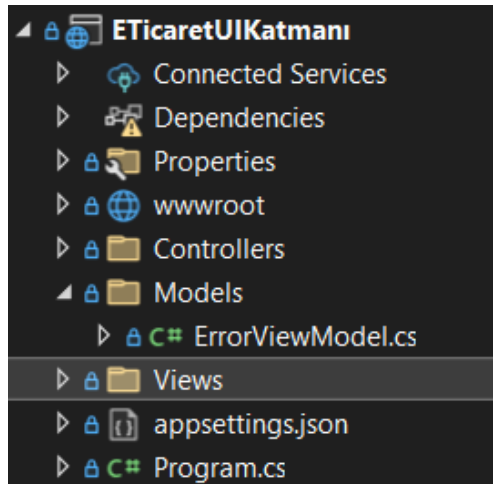
- CSS, JavaScript ve resim dosyalarını içerir.

Controllers: Controller'lar istemciden gelen istekleri işler, iş mantığını yürütür ve uygun veriyi yani sonucu View olarak kullanıcıya döndürür.

- **AccountController.cs:** Kullanıcı giriş, kayıt ve profil işlemlerini yönetir.
- **CartController.cs:** Sepet işlemlerini yönetir.
- **CategoryController.cs:** Kategorileri yönetir.
- **HomeController.cs:** Ana sayfa işlemlerini yönetir.
- **OrderController.cs:** Sipariş işlemlerini yönetir.
- **ProductController.cs:** Ürün işlemlerini yönetir.
- **RoleController.cs:** Kullanıcı rollerini yönetir.
- **UserController.cs:** Kullanıcıları yönetir.

Views: Views, Controller'dan gelen veriyi kullanıcıya sunan ve uygulamanın ön yüzünü oluşturan yapıdır. HTML ve CSS kullanılarak sayfa tasarımları burada oluşturulur.

- **Account:** Kullanıcı giriş ve kayıt sayfaları.
- **Cart:** Sepet görünümü ve ödeme sayfası.
- **Category:** Kategoriler ile ilgili sayfalar.
- **Home:** Ana sayfa.
- **Order:** Sipariş detayları ve sipariş yönetimi.
- **Product:** Ürün detayları ve listesi.
- **Role:** Roller ile ilgili sayfalar.
- **User:** Kullanıcı profili sayfası.
- **Shared:** **_Layout.cshtml** ve **_ViewImports.cshtml** gibi ortak şablonları içerir.
- **Program.cs:** ASP.NET Core uygulamasının başlangıç noktasıdır.



4. Projede Yapılan Ana Geliştirmeler

1. Kimlik Yönetimi (Identity):

- Kullanıcı kaydı, kullanıcı girişi ve rol bazlı yetkilendirme işlemleri yapılmıştır.

2. Sipariş Yönetimi

- Kullanıcıların siparişlerini görüntüleyebileceği Profil sayfası oluşturuldu.
- Sipariş detay sayfası geliştirildi.

3. Sepet Yönetimi

- Kullanıcılar, ürünleri sepete ekleyebilir, çıkarabilir, miktar artırıp azaltabilir.

4. Modern & Kullanıcı Dostu UI

- Bootstrap 5 ve FontAwesome kullanılarak modern bir arayüz tasarlandı.
- Kullanıcı dostu, responsive (mobil uyumlu) bir UI geliştirildi.

5. Sipariş Verme ve Onaylama

- Kullanıcı siteye kayıt yapmadan da sipariş verebilir.
- Sipariş tamamlandıktan sonra uyarı mesajı gösterilir.
- Kayıtlı bir kullanıcı kendi siparişlerini görüntüleyebilir.

6. Admin Paneli

- Admin rolüne sahip kullanıcı veya kullanıcılar; kategorileri, ürünleri, siparişleri, kullanıcıları ve kullanıcıların rollerini yönetebilir.

7. Entity Framework Core ile Veri Yönetimi

- EF Core kullanılarak veritabanı işlemleri gerçekleştirildi.
- Code-First yaklaşımı kullanılarak migration işlemleri yapıldı.

5. Sonuç

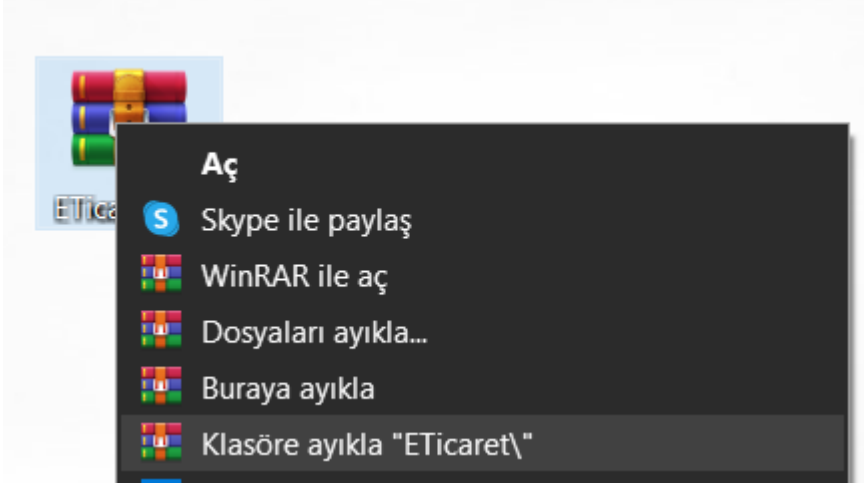
Bu proje, tam özellikli bir e-ticaret platformunun temel işlevlerini gerçekleştirdim. Katmanlı mimari sayesinde kodun okunabilirliği ve sürdürülebilirliği artırmış oldum. ASP.NET Core'un Identity, EF Core, Session Management gibi özellikleri etkin bir şekilde kullandım.

✓ Geliştirmeye açık noktalar:

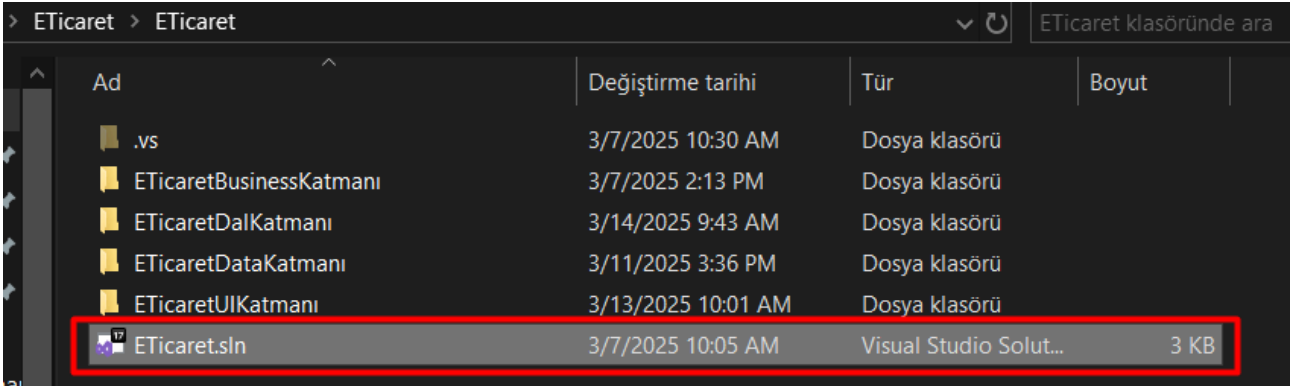
- Ödeme sisteminin entegrasyonu (Stripe, PayPal vb.).
- Kullanıcı değerlendirmeleri (yorum ve puanlama sistemi).
- Admin panelinde gelişmiş raporlama ve analiz.

E-Ticaret Projesinin Adım Adım Çalıştırılması

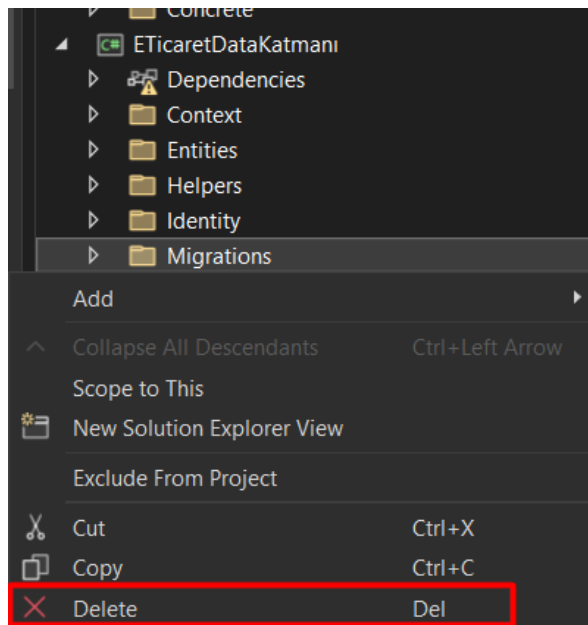
İlk adım olarak, zip dosyasını bir klasöre çıkararak proje dosyalarını klasör halinde açıyoruz.



Ardından, proje klasörü içerisindeki .sln uzantılı dosyaya çift tıklayarak projeyi Visual Studio ile açıyoruz.



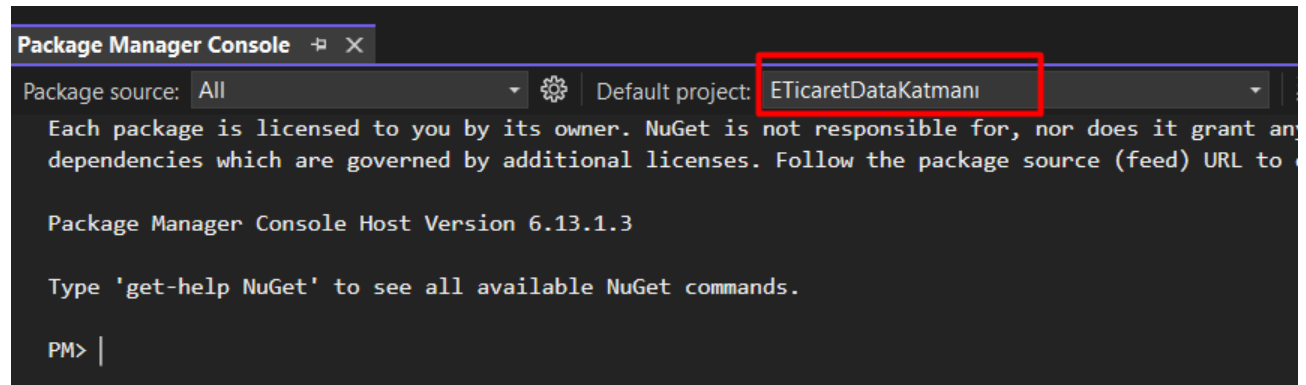
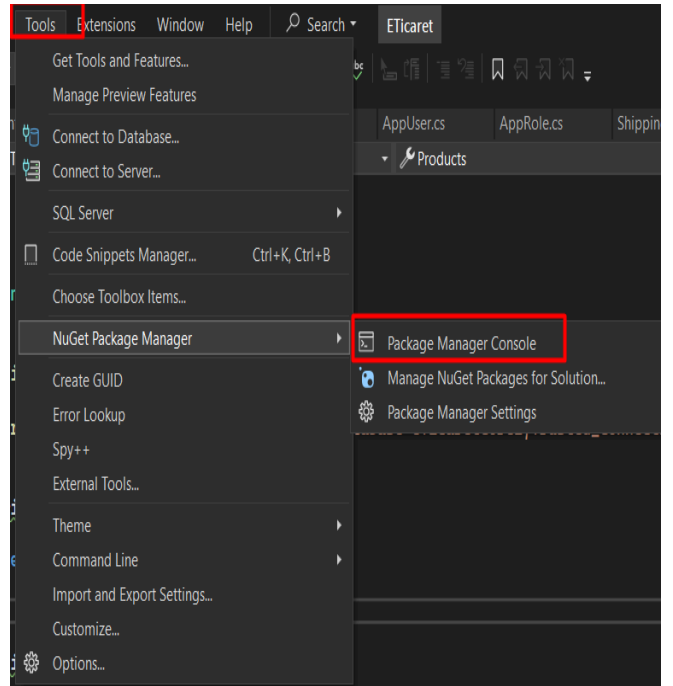
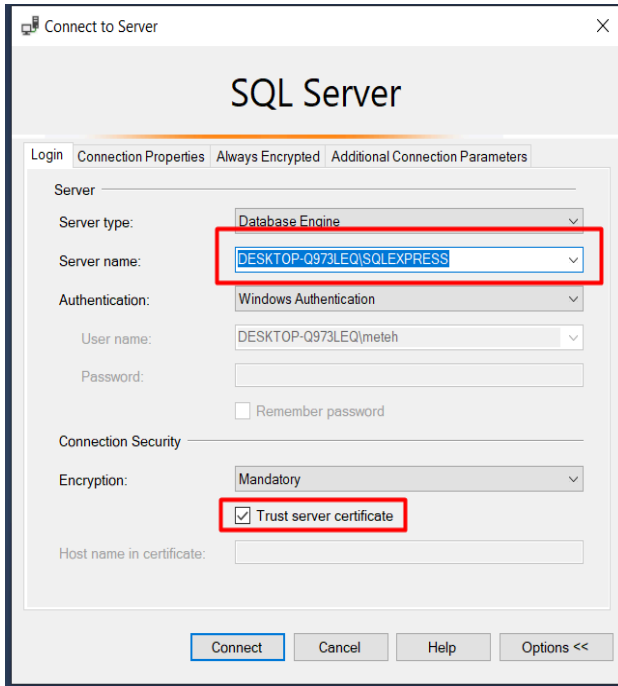
Proje açıldıktan sonra, Data katmanı içerisinde (bazı projelerde farklı bir katmanda da olabilir) Migrations klasörü varsa, bu klasörü tamamen siliyoruz.



Şimdi, Entity'leri veritabanına göndermek için Context.cs dosyasına giderek Connection String değerini projenin kullandığı veritabanına uygun olacak şekilde düzenliyoruz.

```
12 references
public class ETicaretContext : IdentityDbContext<AppUser, AppRole, int>
{
    0 references
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer("Server=DESKTOP-Q973LEQ\\SQLEXPRESS;Database=eTicaretCore2;Trusted_Connection=True;");
    }
    4 references
    public DbSet<Category> Categories { get; set; }
    1 reference
    public DbSet<Order> Orders { get; set; }
    2 references
    public DbSet<Product> Products { get; set; }
    1 reference
    public DbSet<OrderLine> OrderLines { get; set; }
}
```

Server adı, SQL Server Management Studio'da (SSMS) bulunan bilgisayar isminiz ile aynı olmalıdır. Ardından, Visual Studio'da üst menüde bulunan Tools sekmesinden Package Manager Console'u açarak, birkaç komut girerek tabloları veritabanına göndereceğiz.



Package Manager Console'da Default Project olarak ETicaretDataKatmanı (veya projenizdeki ilgili Data Katmanı) seçildikten sonra, sırasıyla aşağıdaki iki komutu gireceğiz.

-> add-migration FirstMigration

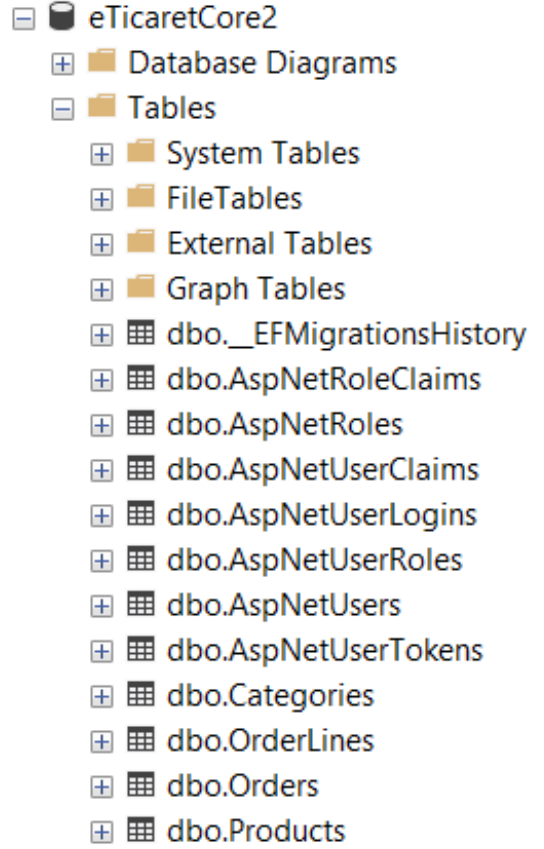
-> update-database

```
PM> add-migration FirstMigration
Build started...
Build succeeded.

No store type was specified for the decimal property 'T
default precision and scale. Explicitly specify the SQL
precision and scale using 'HasPrecision', or configure
No store type was specified for the decimal property 'O
in the default precision and scale. Explicitly specify
specify precision and scale using 'HasPrecision', or co
No store type was specified for the decimal property 'P
default precision and scale. Explicitly specify the SQL
precision and scale using 'HasPrecision', or configure

To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.

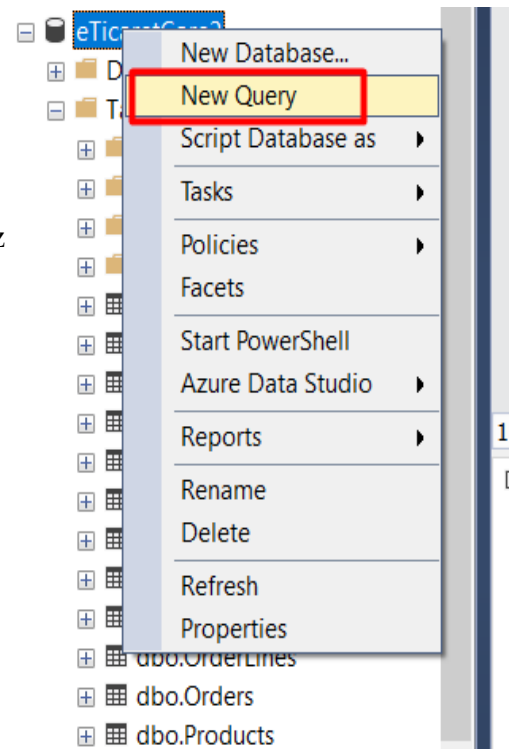
No store type was specified for the decimal property 'T
default precision and scale. Explicitly specify the SQL
precision and scale using 'HasPrecision', or configure
No store type was specified for the decimal property 'O
in the default precision and scale. Explicitly specify
specify precision and scale using 'HasPrecision', or co
No store type was specified for the decimal property 'P
default precision and scale. Explicitly specify the SQL
precision and scale using 'HasPrecision', or configure
Applying migration '20250319201826_FirstMigration'.
Done.
PM>
```



Migration işlemi başarılı bir şekilde tamamlandı ve tablolarımız veritabanına eklendi. Eğer hazır verilerle çalışmak istersek, SSMS üzerinden size verdiğim SQL script'ini çalıştırarak verileri veritabanına ekleyebiliriz. Şimdi adım adım bu işlemi gösterelim. Veri tabanına sağ tık yaparak yeni bir sorgu oluşturuyoruz. Ardından açılan sorgu ekranına script dosyasındaki kodu giriyoruz

```
SQLQuery4.sql - D:\973LEQ\meteh (84)* * * X
USE [eTicaretCore2]
GO
SET IDENTITY_INSERT [dbo].[AspNetRoles] ON
INSERT [dbo].[AspNetRoles] ([Id], [Name], [NormalizedName]
INSERT [dbo].[AspNetRoles] ([Id], [Name], [NormalizedName]
INSERT [dbo].[AspNetRoles] ([Id], [Name], [NormalizedName]
SET IDENTITY_INSERT [dbo].[AspNetRoles] OFF
GO
SET IDENTITY_INSERT [dbo].[AspNetUsers] ON
INSERT [dbo].[AspNetUsers] ([Id], [Name], [Surname], [Use
INSERT [dbo].[AspNetUsers] ([Id], [Name], [Surname], [Use
INSERT [dbo].[AspNetUsers] ([Id], [Name], [Surname], [Use

(1 row affected)
(1 row affected)
(1 row affected)
(1 row affected)
(1 row affected)
(1 row affected)
(1 row affected)
```



Örneğin User Tablosundaki veriler başarılı bir şekilde geldi.

dbo.AspNetRoles	[TwoFactorEnabled]
dbo.AspNetUserClaims	
dbo.AspNetUserLogins	
dbo.AspNetUserRoles	
dbo.AspNetUsers	
dbo.AspNetUserTokens	
dbo.Categories	
dbo.OrderLines	
dbo.Orders	
dbo.Products	

Id	Name	Surname	UserName	NormalizedUserName	Email	NormalizedEmail
1	Metehan	Dünder	EdepHuu	EDEPHUU	metehanndundar@hotmail.com	METEHANNDI
2	Betül	Kamburoglu	betulkamburoglu	BETULKAMBUROGLU	betulkamburoglu@gmail.com	BETULKAMBL

Ardından direkt olarak programı çalıştırabiliriz.

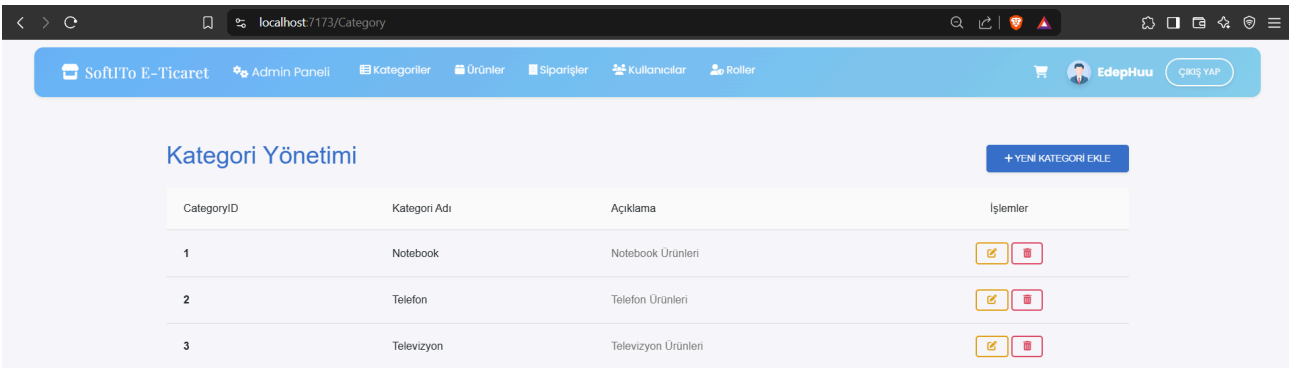
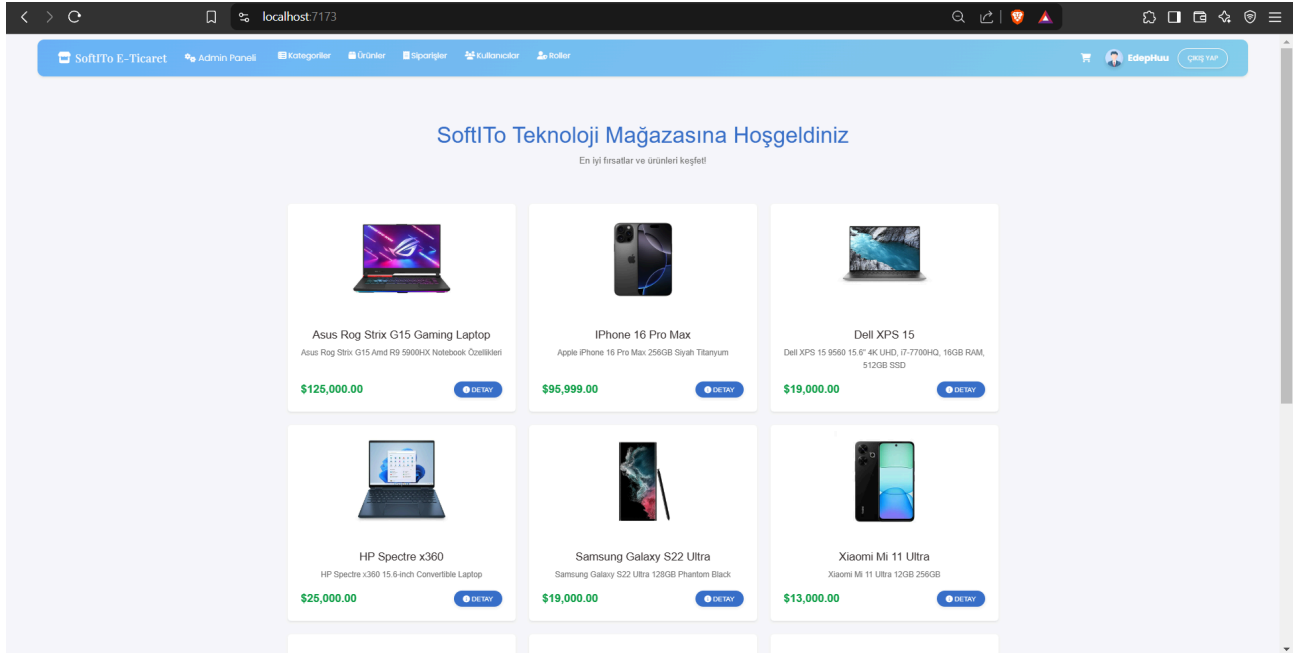
Admin Girişi:

- Kullanıcı Adı: EdepHuu / Sifre: Admin123.

Müşteri Girişi

- Kullanıcı Adı: betulkamburoglu / Sifre: Betul123.

Admin Paneli Arayüzü



SoftTo E-Ticaret

Admin Paneli

Kategoriler

Ürünler

Siparişler

Kullanıcılar

Roller

EdepHuu

ÇIKIŞ YAP

Ürün Yönetimi

+YENİ ÜRÜN EKLE

ProductID	Ürün Adı	Kategori	Görsel	Stok	Fiyat	Öne Çıkan	Onaylı	İşlemler
1	Asus Rog Strix G15 Gaming Laptop	Notebook		94	125000.00 ₺	Evet	Onaylı	
2	IPhone 16 Pro Max	Telefon		44	95999.00 ₺	Evet	Onaylı	
3	Dell XPS 15	Notebook		44	19000.00 ₺	Evet	Onaylı	
4	HP Spectre x360	Notebook		57	25000.00 ₺	Evet	Onaylı	
5	Samsung Galaxy S22 Ultra	Telefon		75	19000.00 ₺	Evet	Onaylı	
6	Xiaomi Mi 11 Ultra	Telefon		37	13000.00 ₺	Evet	Onaylı	
7	Samsung QLED 8K	Televizyon		29	20000.00 ₺	Evet	Onaylı	

SoftTo E-Ticaret

Admin Paneli

Kategoriler

Ürünler

Siparişler

Kullanıcılar

Roller

EdepHuu

ÇIKIŞ YAP

Siparişlerim

87d6b4aab2d04bf68b552f247bbd9f73

Müşteri Kullanıcı Adı: Mustafa Kemal

Sipariş Tarihi: 3/21/2025

\$60,000.00

Waiting

ONAYLA

SİL

DETAYLAR

c018a888cf1d411f8b9950b3c925160a

Müşteri Kullanıcı Adı: Metehan Dündar

Sipariş Tarihi: 3/21/2025

\$44,000.00

Waiting

ONAYLA

SİL

DETAYLAR

d8e1eca5f11e41ba9fc756afed6bfd86

Müşteri Kullanıcı Adı: betulkamburoglu

Sipariş Tarihi: 3/21/2025

\$162,000.00

Completed

ONAYLA

SİL

DETAYLAR

SoftTo E-Ticaret

Admin Paneli

Kategoriler

Ürünler

Siparişler

Kullanıcılar

Roller

EdepHuu

ÇIKIŞ YAP

Kullanıcı Yönetimi

Sistemde kayıtlı olan kullanıcıları yönetebilirsiniz.

Kullanıcı Adı	İşlemler
Betul	ROLATA SİL

SoftTo E-Ticaret

Admin Paneli

Kategoriler

Ürünler

Siparişler

Kullanıcılar

Roller

EdepHuu

ÇIKIŞ YAP

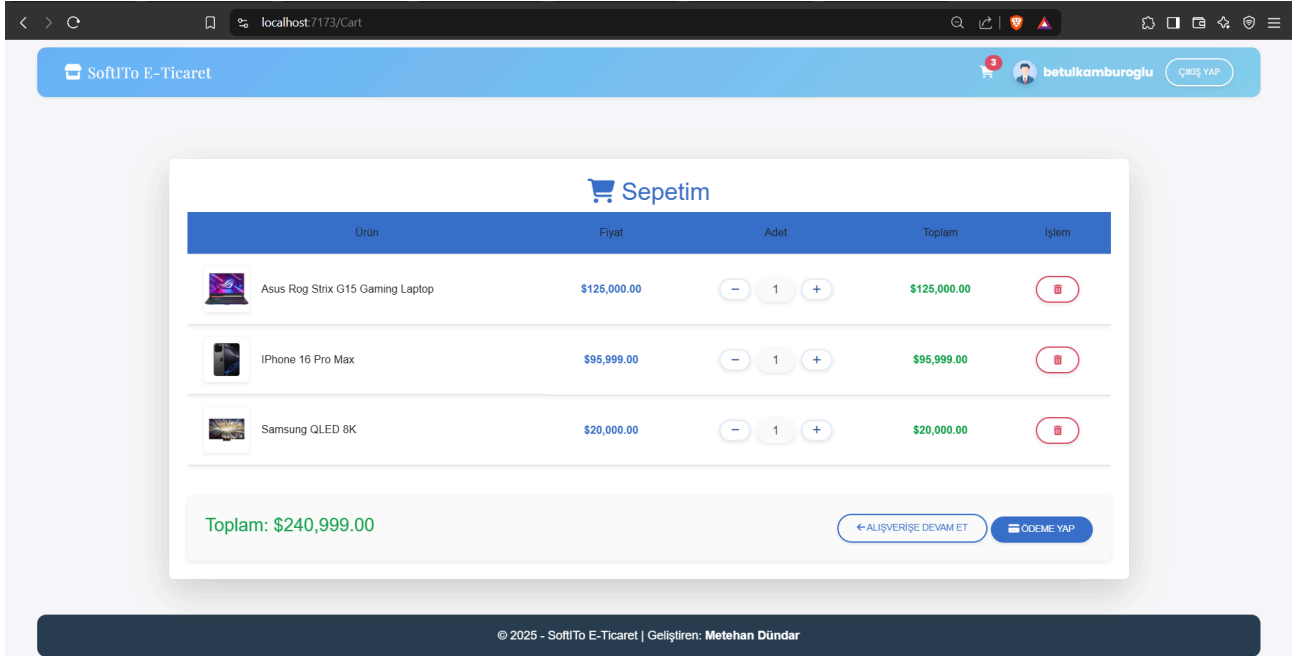
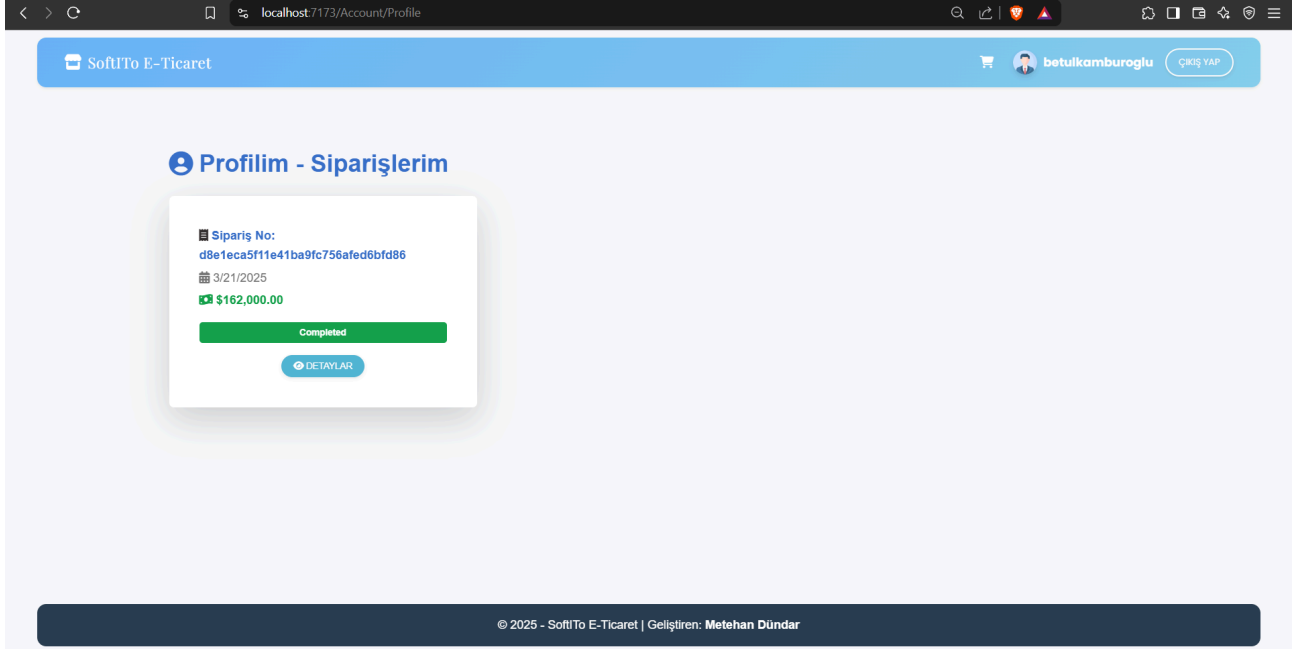
Roller

+YENİ ROL EKLE

#	Rol Adı	İşlemler
1	User	DÜZENLE SİL

Admin panelinde, **Ürünler, Kategoriler, Kullanıcılar ve Roller** için **ekleme, silme ve düzenleme** işlemleri gerçekleştirilebilir. Ayrıca, **siparişler** görüntülenebilir, **ayrıntıları** incelenebilir ve **onaylama** işlemi yapılabilir.

Müşteri Kullanıcı Arayüzü



Müşteri, **sipariş oluşturabilir** ve **siparişlerinin onay durumunu** profil sayfasından görüntüleyebilir.